

React Full Notes + 30 Practice Problems (Beginner to Advanced)

This PDF contains React notes from very basic to advanced level along with 30 interview-style coding problems for frontend developers.

1. What is React?

React is a JavaScript library used to build user interfaces using reusable components.

2. Why React?

React provides reusable components, virtual DOM, better performance, state management, and component-based architecture.

3. JSX

```
const element = <h1>Hello React</h1>;
```

4. Components

```
function App() {  
  return <h1>Hello</h1>  
}
```

5. Functional Components

Modern React applications mostly use functional components.

6. Props

```
function User(props) {  
  return <h1>{props.name}</h1>  
}
```

7. State

```
const [count, setCount] = useState(0);
```

8. useState Hook

Used to manage component state.

9. Event Handling

```
<button onClick={handleClick}>Click</button>
```

10. Conditional Rendering

```
{isLoggedIn ? <Home /> : <Login />}
```

11. List Rendering

```
users.map(user => <li>{user.name}</li>)
```

12. Keys in React

Keys help React identify changed elements efficiently.

13. useEffect Hook

```
useEffect(() => {  
  fetchData();  
}, []);
```

14. Controlled Components

Form inputs controlled using state.

15. Lifting State Up

Sharing state between components using parent component.

16. Prop Drilling

Passing props through multiple nested components.

17. Context API

Used to avoid prop drilling.

18. useRef Hook

Used to directly access DOM elements and persist values.

19. useMemo Hook

Optimizes expensive calculations.

20. useCallback Hook

```
Memoizes functions to avoid unnecessary re-renders.
```

21. React.memo

Prevents unnecessary component re-renders.

22. Custom Hooks

Reusable logic using hooks.

23. React Router

Used for routing in React applications.

24. Lazy Loading

Loads components only when needed.

25. API Calls

```
useEffect(() => {  
  fetch('https://api.com')  
}, [])
```

26. Forms in React

Controlled and uncontrolled forms.

27. Redux Basics

Global state management library.

28. Performance Optimization

Memoization, lazy loading, code splitting, virtualization.

29. React Lifecycle

Mounting, updating, unmounting phases.

30. Real World React Usage

Dashboards, ecommerce apps, admin panels, portfolios, SaaS products.

30 React Practice Problems

- 1 Create a counter app using useState.
- 2 Build a toggle button component.
- 3 Create a live character counter.
- 4 Build a todo app.
- 5 Create a search filter functionality.
- 6 Build a password show/hide feature.
- 7 Create a reusable button component.
- 8 Build an accordion component.
- 9 Create tabs component.
- 10 Build a modal popup.
- 11 Create a dropdown menu.
- 12 Build pagination component.
- 13 Create dark mode toggle.
- 14 Build a form validation app.
- 15 Create a stopwatch app.
- 16 Build a countdown timer.
- 17 Create a digital clock.
- 18 Build weather app using API.
- 19 Create infinite scroll functionality.
- 20 Build shopping cart logic.
- 21 Create reusable input component.
- 22 Build authentication UI.
- 23 Create protected routes.
- 24 Build CRUD app with API.
- 25 Create custom hook for fetching data.
- 26 Build debounce search input.
- 27 Create drag and drop list.
- 28 Build image slider.
- 29 Create theme switcher using Context API.
- 30 Build admin dashboard layout.

Most Asked React Interview Questions

- What is Virtual DOM?
- Difference between state and props

- What are hooks?
- Difference between useEffect and useLayoutEffect
- What is prop drilling?
- What is Context API?
- Difference between controlled and uncontrolled components
- What is React.memo?
- Difference between useMemo and useCallback
- What are custom hooks?
- What is reconciliation?
- Why keys are important in React?
- What causes re-renders in React?
- How React lifecycle works?
- What is lazy loading?
- What is code splitting?
- How to optimize React performance?
- Difference between Redux and Context API
- What are higher-order components?
- What is hydration in React?